

实验报告：控制一个可清理的后台任务

项目	内容
课程名称	系统开发工具基础
实验主题	命令行环境：进程、信号与任务控制
题目编号	第5题
学号	25020007191
姓名	周御基
实验日期	2026年8月31日

一、实验目的

1. 掌握 Shell 任务控制 (job control)：前台启动、`Ctrl-Z` 挂起、`bg` 后台运行、`jobs` 查看状态
2. 学会不手工抄写 PID 的情况下获取进程号 (`jobs -p`)
3. 掌握发送 `SIGTERM` 优雅终止进程，并验证 `trap` 清理动作
4. 理解信号处理机制 (`trap`) 与前台/后台任务切换

二、实验环境

- 操作系统：Ubuntu Linux (远程服务器 192.168.79.130, Ubuntu 25.04)
- 命令行工具：Bash (PTY 交互终端)
- 远程连接：SSH (paramiko, 带伪终端)

三、实验步骤与代码

步骤1：创建 worker.sh

按题目给定内容创建 `~/q05/worker.sh`：

```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
```

脚本用 `trap` 捕获 `TERM` (`SIGTERM`) 和 `INT` (`SIGINT`) 信号, 收到信号时向 `cleanup.log` 追加 `CLEAN_EXIT` 并以状态 0 退出。

赋予执行权限并清理旧文件、启动前先结束残留进程:

```
cd ~/q05
chmod +x worker.sh
pkill -f worker.sh 2>/dev/null; sleep 1
rm -f stdout.log stderr.log cleanup.log
```

输出结果:

(无输出 · 执行成功)

步骤2: 前台启动, 把 `stdout`、`stderr` 分别写入文件

```
./worker.sh > stdout.log 2> stderr.log
```

输出结果:

(前台运行 · 每隔 1 秒计数 · `stdout` 写入 `stdout.log` · `stderr` 写入 `stderr.log`)

步骤3: 使用 `Ctrl-Z` 挂起任务, 再用 `bg` 后台继续

```
# 按 Ctrl-Z ( 发送 SIGTSTP 信号 )
^Z
# 用 bg 让它在后台继续
bg
```

输出结果:

```
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
```

`Ctrl-Z` (`^Z`) 发送 `SIGTSTP` 信号挂起前台任务 (状态从 `Running` 变为 `Stopped`) ; `bg` 让被挂起的任务在后台继续运行。

步骤4: 使用 `jobs` 确认状态

```
jobs -l
```

输出结果:

```
[1]+  53503 Running              ./worker.sh > stdout.log 2> stderr.log &
```

`jobs -l` 显示任务编号 `[1]`、进程号 `53503` 以及当前状态 `Running`。

步骤5: 用 `jobs -p` 取得 PID (不手工抄写)

```
PID=$(jobs -p)
echo "PID=$PID"
kill $PID
```

输出结果:

```
PID=53503
```

用 `$(jobs -p)` 把任务号对应的进程号赋给变量, 全程未手工输入 PID, 符合题目要求。

步骤6: 发送 `SIGTERM` 并等待任务结束

```
kill $PID
wait
```

输出结果:

(无输出 · 任务收到 `SIGTERM` 后执行 `trap` 清理并以 `0` 退出)

`kill` 默认发送 `SIGTERM` (不是 `kill -9`) , 进程的 `trap` 处理器得以执行。

步骤7：验证清理与退出

检查是否还有 `worker` 进程残留，并查看 `stdout.log` 与 `cleanup.log`：

```
pgrep -af worker.sh || echo NO-WORKER
head -3 stdout.log
wc -l cleanup.log && cat cleanup.log
```

输出结果：

```
NO-WORKER
0
1
2
1 cleanup.log
CLEAN_EXIT
```

四、实验结果

4.1 结果展示

- `cleanup.log` 中出现恰好 1 行 `CLEAN_EXIT` , 证明 `trap` 清理动作被触发
- `pgrep -af worker.sh` 显示 `NO-WORKER` , 证明任务已退出、无残留
- `stdout.log` 记录了前台运行阶段的输出 `0 1 2`
- PID 通过 `jobs -p` 自动获取，未手工抄写

4.2 结果分析

- 作业控制流程：前台启动 → `Ctrl-Z` (`SIGTSTP`, 挂起) → `bg` (后台继续) → `jobs -l` 查看 PID 与状态，是 Shell 任务控制的标准流程。
- `kill` 默认发送 `SIGTERM`，进程的 `trap '... TERM INT'` 处理器会先执行清理动作（写入 `CLEAN_EXIT`）再退出，体现了“优雅终止”（graceful shutdown）的意义。
- 若用 `kill -9` 则无法执行 `trap`，`cleanup.log` 不会出现 `CLEAN_EXIT`，故题目特别要求“禁止使用 `kill -9`”。

五、实验总结

通过本次实验，我掌握了以下技能：

1. 用 `Ctrl-Z` / `bg` / `jobs` 进行 Shell 任务控制
 2. 用 `jobs -p` 不手工抄写地获取后台任务 PID
 3. 用 `kill` (SIGTERM) 优雅终止进程，配合 `trap` 完成清理
 4. 验证进程退出状态与清理结果 (`pgrep` 、日志检查)
-

实验中的脚本文件：

1. `Scripts/q05/worker.sh`